

JurisMon — System Handover & Operations Guide

Audience: System operators, maintainers, and incoming developers.

Scope: Architecture, deployment, ingestion pipeline, configuration, migrations, admin management, and known limitations.

1. System Architecture

JurisMon is an automated regulatory drift monitoring platform that crawls municipal and county government portals, extracts text, computes structural clause differences (§ sections and paragraphs), persists historical snapshots, and serves full-text search and subscription management.

[Government & Municipal Portals]

|



[1. Crawler Pipeline]

(crawler/orchestrator.py)

└─ crawler/requests_crawler.py

└─ crawler/playwright_crawler.py

└─ crawler/adapters/ (municode, granicus, civicplus, rest_api, etc.)

|



[2. Extraction Pipeline]

(extractor/cleaner.py, extractor/pdf_extractor.py, extractor/html_extractor.py)

└─ Boilerplate & nav stripping

└─ Tesseract OCR fallback (extractor/ocr.py)

|



[3. Diff Engine]

(diff_engine/engine.py, diff_engine/section_splitter.py)

└─ Structural clause identification (§ sections, code numbers, paragraphs)

└─ SHA-256 content hashing & 3-way delta detection (added, removed, modified)

|



[4. Persistence Layer]

(db/repository.py, db/client.py)

└─ Cloud PostgreSQL via Supabase PostgREST client (production)

└─ Local SQLite fallback (local dev / test environments)

|



[5. Application & Search API]

(api/main.py, api/auth.py, notifications/mailer.py)

└─ FastAPI REST endpoints (/api/search, /api/auth/*, /api/admin/*)

└─ PayPal Subscriptions & Webhook Handler (api/payments/paypal_provider.py)

|



[6. Presentation Layer]

└─ frontend/index.html (Search UI, diff viewer, 14-day trial gating)

└─ frontend/admin.html (Source health monitor, crawl triggers, subscribers)

└─ frontend/terms.html (Terms of Service & statutory disclaimers)

└─ frontend/privacy.html (Privacy Policy, named processors, GDPR rights)

Core Modules

- `crawler/orchestrator.py` : Coordinates daily crawl jobs, concurrency limits, delays, and routes URLs to platform adapters.
- `crawler/adapters/` : Platform-specific logic for municipal software (`municode.py` , `granicus.py` , `civicplus.py` , `rest_api.py` , `xml_feed.py` , `direct_document.py` , `custom.py`).
- `extractor/` : Cleans HTML and PDF documents, strips headers/footers, extracts text, and triggers OCR when necessary.
- `diff_engine/engine.py` : Splits extracted legal texts into granular clauses and compares current text against prior baseline snapshots.
- `db/repository.py` : Unified database access layer supporting both Supabase (PostgreSQL) and SQLite.
- `api/main.py` : FastAPI server handling search queries, authentication, crawl status, admin endpoints, and static HTML routes.
- `api/auth.py` : Bcrypt password hashing, JWT issue/validation, rate limiting, and role-based permissions (`admin` vs `customer`).
- `api/payments/paypal_provider.py` : Verifies PayPal webhook signatures and maps subscription transactions to user entitlements.
- `notifications/mailer.py` : Transactional email dispatch via Resend API (signup confirmation codes, password reset codes).

2. Deployment Workflow

⚠ **CRITICAL RULE:** Running `git push` to GitHub does **NOT** update the live site at `https://jurismo.n.com` .

Deployment is a manual, isolated operational step on the production VPS.

Deployment Instructions

Detailed deployment documentation is maintained in [README.md](#).

Summary command to run as root on the VPS:

```
sudo bash /var/www/jurismon/scripts/deploy.sh
```

This script: 1. Hard-resets `/var/www/jurismon` to `origin/main` . 2. Installs updated Python requirements in `/var/www/jurismon/venv` . 3. Verifies Playwright Chromium dependencies. 4. Validates required environment variables in `/var/www/jurismon/.env` . 5. Syncs systemd service and timer units, and reloads systemd. 6. Replaces and reloads Nginx site configurations. 7. Restarts `jurismon.service` and verifies that the API responds on `127.0.0.1:8000` .

To verify active deployment:

```
git -C /var/www/jurismon rev-parse --short HEAD
systemctl status jurismon.service
systemctl status jurismon-crawl.timer
```

3. Daily Ingestion & Crawl Operations

The ingestion pipeline checks monitored jurisdictions daily for new meeting minutes, zoning changes, and statutory revisions.

- **Systemd Timer:** `jurismon-crawl.timer`
- **Systemd Service:** `jurismon-crawl.service`
- **Schedule:** Daily at `04:00:00 UTC` with a randomized delay of up to 30 minutes (`RandomizedDelaySec=1800`, `Persistent=true`).
- **Log Inspection:** ```bash # View live trailing logs journalctl -u jurismon-crawl.service -f`

`# View the last 100 log lines journalctl -u jurismon-crawl.service -n 100 --no-pager ```

Manual Crawl Execution

You can trigger a crawl manually at any time through two methods:

1. **From the Admin Web Panel:** - Log into `https://jurismon.com/admin` . - Click the "Trigger Live Crawl" button. - The API issues a non-blocking `systemctl start --no-block jurismon-crawl.service` call using scoped sudoers permissions (`/etc/sudoers.d/jurismon-crawl`).
2. **From the VPS Shell:** ```bash # Option A: Run via systemd sudo systemctl start jurismon-crawl.service`

`# Option B: Run directly in foreground as application user sudo -u jurismon
/var/www/jurismon/venv/bin/python /var/www/jurismon/scripts/run_crawler.py ```

4. Environment Variables Reference

All runtime configuration is managed via `/var/www/jurismon/.env` (permissions `0600`, owned by `jurismo` `n:jurismon`). No literal credentials should ever be committed to git.

Variable Name	Purpose	Required in Production
<code>APP_ENV</code>	Application environment mode (<code>development</code> or <code>production</code>). Production enforces strict credential checks.	Yes
<code>SECRET_KEY</code>	Cryptographic secret used for signing and verifying customer JWT access tokens.	Yes
<code>ADMIN_EMAIL</code>	Email address permitted to access the <code>/admin</code> portal.	Yes
<code>ADMIN_PASSWORD_HASH</code>	Salted bcrypt hash of the admin password. The API fails to start if empty in production.	Yes
<code>CORS_ALLOWED_ORIGINS</code>	Comma-separated list of allowed HTTP origins. Must include <code>https://jurismon.com</code> . Never use <code>*</code> .	Yes
<code>SUPABASE_URL</code>	PostgREST endpoint URL of the Supabase PostgreSQL instance.	Yes
<code>SUPABASE_KEY</code>	Supabase service role secret key for database operations.	Yes
<code>DATABASE_URL</code>	PostgreSQL connection string (<code>postgresql://...</code>) used for applying schema migrations via psql.	Required for migrations
<code>PAYPAL_MODE</code>	PayPal environment (<code>sandbox</code> or <code>live</code>).	Yes
<code>PAYPAL_CLIENT_ID</code>	Client ID from the PayPal Developer Portal.	Yes
<code>PAYPAL_CLIENT_SECRET</code>	Client secret from the PayPal Developer Portal.	Yes
<code>PAYPAL_WEBHOOK_ID</code>	Webhook identifier registered in PayPal. Signature verification fails closed if mismatched.	Yes
<code>PAYPAL_PRODUCT_ID</code>	Reference PayPal Product ID for the catalog.	Reference
<code>PAYPAL_PLAN_ID_MONTHLY</code>	Active PayPal Billing Plan ID for monthly billing (\$49/mo).	Reference
<code>PAYPAL_PLAN_ID_ANNUAL</code>	Active PayPal Billing Plan ID for annual billing (\$468/yr).	Reference
<code>RESEND_API_KEY</code>	API token for Resend transactional email delivery.	Yes
<code>MAIL_FROM</code>	Sender address for transactional emails (e.g. <code>JurisMon <noreply@jurismon.com></code>).	Yes

Variable Name	Purpose	Required in Production
<code>ALERT_EMAIL</code>	Destination address for crawl health summaries and crawler failure alerts.	No (defaults to <code>ADMIN_EMAIL</code>)
<code>ALERT_ON EVERY CRAWL</code>	Set to <code>true</code> to send crawl reports on every run; <code>false</code> alerts only on errors.	No (defaults to <code>false</code>)
<code>CRAWLER_CONCURRENCY</code>	Maximum concurrent portal requests during a crawl run (default: <code>3</code>).	No
<code>CRAWLER_DELAY_SECONDS</code>	Polite delay between successive requests to the same domain (default: <code>1.5</code> s).	No
<code>CRAWLER_USER_AGENT</code>	User-Agent string sent during HTTP discovery and extraction.	No
<code>HEADLESS_BROWSER</code>	Enables Playwright Chromium for dynamic JavaScript portal extraction (<code>true</code> / <code>false</code>).	No (defaults to <code>true</code>)
<code>STRIPE_API_KEY</code>	Optional / pluggable Stripe provider key for future billing options.	No
<code>STRIPE_WEBHOOK_SECRET</code>	Optional / pluggable Stripe webhook verification secret.	No

5. Database Migrations & Schema Parity Guard

Migration Files

All schema migrations are stored sequentially in `db/migrations/`:

1. `001_initial_schema.sql`: Initial sources, documents, snapshots, diffs, and crawl runs.
2. `002_subscriptions.sql`: PayPal subscriptions and webhook event logs.
3. `003_customer_auth.sql`: Customer users, email verification codes, and trial timestamps.
4. `004_fix_subscriptions_schema.sql`: Normalizes `subscriptions.user_id` and adds indexes.
5. `005_password_resets.sql`: Adds password reset indices and verification token expiry handling.
6. `006_crawl_sources_skipped.sql`: Adds `sources_skipped` column to `crawl_runs`.

Applying Migrations

Apply migrations using the runner script:

```
/var/www/jurismon/venv/bin/python /var/www/jurismon/scripts/run_migrations.py
```

Alternatively, execute the idempotent `.sql` files directly in the Supabase SQL editor or via `psql`.

Schema Parity Guard (`tests/test_schema_contract.py`)

⚠ **Historical Defect Warning:** The application was built with SQLite as a local fallback and PostgreSQL (Supabase) for production. On three separate occasions, new columns were added to SQLite without a matching PostgreSQL migration script, causing production database queries to fail during live execution (most recently with `crawl_runs.sources_skipped`).

To permanently prevent this, `tests/test_schema_contract.py::TestSQLitePostgresSchemaParity` inspects the live SQLite schema generated by `db/repository.py` and verifies every table and column against `db/migrations/*.sql` . If a developer adds a column to SQLite without authoring an equivalent migration, the test suite halts execution and refuses to pass.

6. Known System Limitations

- In-Memory Rate Limiting:** - The rate limiters for login attempts and forgot-password requests are maintained in memory (`api/main.py`). They reset when `jurismon.service` restarts and are scoped to the running process. For the current single-worker deployment this is fully sufficient, but multi-worker scaling requires migrating rate-limit counters to Redis or database persistence.
- Source Ingestion Coverage (24 of 65 Inactive):** - Out of the 65 monitored jurisdictions in `config/sites.json` , 41 are operational and crawled daily. 24 sources are currently un-ingestible due to external municipal blockers (dead links, Cloudflare Turnstile bot blocks, HTTP 403 blocks, or portal authentication requirements). - See [docs/SOURCE_COVERAGE.md](#) for the exact source-by-source status breakdown.
- Diffs Require a Baseline Snapshot:** - Clause deltas (added/removed text) are only generated on the **second** and subsequent crawls of any document. The initial crawl establishes the baseline snapshot. The absence of diffs on a brand-new source is expected behavior, not a defect.
- Municipal Portal Markup Changes:** - Municipalities periodically redesign their website layouts, change CMS systems, or update meeting notice formats. Adapters in `crawler/adapters/` require ongoing maintenance when a city alters its portal layout.

7. Admin Credentials & Access Rotation

The administrative panel (`https://jurismon.com/admin`) is protected by JWT Bearer authentication: `POST /api/admin/login` validates the submitted password against `ADMIN_PASSWORD_HASH` and issues a signed bearer token, `api/auth.py` enforces security via `HTTPBearer` with `require_admin()` decoding the JWT claims, and the frontend persists the authenticated session in `localStorage` under `jurismon_admin_token` .

Password Rotation Procedure

To change or rotate the admin password:

1. **Generate a new bcrypt password hash:** `bash /var/www/jurismon/venv/bin/python /var/www/jurismon/scripts/generate_admin_hash.py` This script securely prompts for a new password (or generates a high-entropy random password) and prints only the resulting bcrypt hash.
 2. **Update the production environment file:** Edit `/var/www/jurismon/.env` and update: `bash ADMIN_PASSWORD_HASH="<new-bcrypt-hash>"`
 3. **Restart the API service:** `bash sudo systemctl restart jurismon.service`
 4. **Verify authentication:** - Log in at `https://jurismon.com/admin` using the new password. - Confirm that authentication fails with the previous password. - **Never write, print, or commit plaintext passwords or hashes to documentation, tickets, or git commits.**
-

8. Running the Automated Test Suite

The test suite covers database abstraction, full-text search, customer auth, payment webhooks, schema parity, crawler adapters, and legal page endpoints:

```
# Activate virtual environment
source /var/www/jurismon/venv/bin/activate # Or on local: .\venv\Scripts\activate

# Run full test suite
python -m pytest tests/ -v
```

All 252 test cases run completely self-contained and offline: - Tests use temporary SQLite databases or mocked PostgREST clients. - External services (PayPal, Supabase cloud, Resend) are mocked in integration tests to ensure deterministic execution without live API credentials.